

Day 41: Read Transactions from Text File

Day 41 — Read Transactions from Text File

1. Day Number

Day 41

2. Topic Name

Basic File Reading

Today you will learn how Python reads information that has already been saved inside a text file.

3. Connection with Yesterday

Yesterday, on **Day 40**, you created transaction history and wrote those transactions into a text file.

The flow was:

```
Python list
  ↓
write transactions
  ↓
transactions.txt
```

Today you will do the opposite:

```
transactions.txt
  ↓
read file
  ↓
Python program
  ↓
print transactions
```

So the basic lifecycle is becoming:

```
Create Data
  ↓
Store Data
  ↓
Read Data
  ↓
Use Data
```

This is an important foundation for later working with logs, configuration files, CSV files, databases, and persistent application data.

4. Important Topics

Today focus on:

- Read mode "r"
- open()
- with
- Looping through file lines
- strip()
- Basic idea of file-not-found errors

The important pattern is:

```
with open("filename.txt", "r") as file:  
    # read from file
```

Do not worry about memorizing it immediately. Understand what each part means first.

5. Foundational Notes

What does reading a file mean?

Suppose transactions.txt contains:

```
Deposit: 500  
Withdraw: 200  
Payment: 100
```

Your Python program can open this file and retrieve these values.

Conceptually:

```
Hard Disk  
  |  
  | transactions.txt  
  ↓  
Python Program  
  |  
  ↓  
Read each line
```

open()

Python provides the built-in open() function for working with files.

General structure:

```
open("filename", "mode")
```

For example:

```
open("transactions.txt", "r")
```

Here:

```
transactions.txt → file name  
"r"              → read mode
```

What is "r" mode?

"r" means:

| Open the file for reading.

Think:

```
r = read
```

You are telling Python:

```
I only want to read information from this file.
```

Unlike "w" mode from yesterday, "r" does **not** erase the existing file contents.

Why use with?

You will commonly see:

```
with open(...) as file:
```

The `with` statement handles opening and closing the file safely.

Conceptually:

```
with  
↓  
Open file  
↓  
Use file  
↓  
Automatically close file
```

Without `with`, you would have to remember to close the file manually.

For beginners, prefer:

```
with open(...)
```

A text file can be looped through

A file containing:

```
Deposit 500
Withdraw 100
Payment 50
```

can be processed line by line.

Conceptually:

```
for each line in file
    ↓
    process the line
```

So if there are 3 transaction lines, the loop runs 3 times.

Important: lines usually contain `\n`

Suppose your file visually contains:

```
Deposit 500
Withdraw 100
```

Internally, Python may see something similar to:

```
"Deposit 500\n"
"Withdraw 100\n"
```

`\n` represents a newline.

If you directly print a line that already contains `\n`, `print()` adds another newline of its own.

That can produce output such as:

```
Deposit 500

Withdraw 100
```

Notice the unwanted blank line.

`strip()`

`strip()` helps remove unnecessary whitespace around a string.

Example:

```
text = "  Deposit 500\n"
```

After using:

```
text.strip()
```

the result becomes conceptually:

```
"Deposit 500"
```

So:

```
"Deposit 500\n"  
    ↓ strip()  
"Deposit 500"
```

This is why `strip()` is useful when reading lines from text files.

6. Easy Example

Suppose a file named:

```
colors.txt
```

contains:

```
Red  
Blue  
Green
```

You could conceptually read it using:

```
with open("colors.txt", "r") as file:  
    for line in file:  
        color = line.strip()  
        print(color)
```

Output:

```
Red  
Blue  
Green
```

Here the important flow is:

```
Open file  
    ↓  
Get one line  
    ↓  
Remove newline using strip()  
    ↓  
Print clean line  
    ↓  
Repeat
```

This example shows the reading technique, but it is not the solution to today's transaction problem.

7. Problem Statement

Create a Python program that:

Reads transaction lines from a text file and prints each transaction without extra newline spaces.

Assume the transaction file may contain:

```
Deposit: 500  
Withdraw: 200  
Payment: 100
```

Your program should read the file one line at a time.

For every line:

1. Remove unnecessary newline/whitespace.
2. Print the cleaned transaction.

8. Concepts Used

You will use:

File handling

```
open()
```

to open the transaction file.

Read mode

```
"r"
```

to tell Python that the file should be opened for reading.

Context manager

```
with
```

to safely manage the file.

Loop

```
for
```

to process each transaction line.

String method

```
strip()
```

to remove extra whitespace and newline characters.

Function

You should place the file-reading logic inside a function.

Conceptually:

```
main program
  ↓
read_transactions(...)
  ↓
open file
  ↓
read lines
  ↓
print transactions
```

9. Thought Process

Before coding, think through the problem.

Step 1: Where is the information?

The transactions are no longer only inside a Python list.

They are stored inside:

```
transactions.txt
```

So you first need to open the file.

Step 2: Which mode should be used?

You only want to read existing information.

Therefore:

```
read mode → "r"
```

Step 3: How should transactions be processed?

Each transaction is stored on a separate line.

Therefore, looping through the file is a natural approach:

```
transaction 1
transaction 2
transaction 3
```

Process them one at a time.

Step 4: Why use strip()?

Each line may contain:

```
\n
```

at the end.

So clean the line before printing it.

```
line
↓
strip()
↓
clean transaction
```

Step 5: What happens if the file is empty?

If the file contains no lines:

```
transactions.txt
-----
nothing
-----
```

the loop simply has nothing to process.

For today's beginner problem, that is acceptable.

Step 6: What happens if the file does not exist?

If Python tries to read:

```
transactions.txt
```

but that file does not exist, Python can raise:

```
FileNotFoundError
```

You do not need advanced error handling yet.

For now, understand the idea:

```
Python asks for file
      ↓
Does file exist?
  /      \
Yes       No
  ↓       ↓
Read     FileNotFoundError
```

Later you can learn try and except to handle this cleanly.

10. Pseudocode

```
FUNCTION read_transactions(file_name)

    OPEN file_name in read mode
```



```

    FOR each line in file
        remove extra whitespace/newline from line
        print cleaned transaction
    END FOR
END FUNCTION

CALL read_transactions with transaction file name

```

Notice that the pseudocode focuses on the logic rather than Python syntax.

11. Suggested Solving Approach

Use a **functional approach**.

Instead of putting everything directly into the main program, create a function responsible for reading the transactions.

Conceptually:

```
read_transactions(filename)
```

Responsibility:

```

Input:
    file name
    ↓
Function:
    open file
    read lines
    clean lines
    print transactions
    ↓
Output:
    transactions displayed

```

This keeps one responsibility inside one function.

For example:

```

read_transactions()
|
├─ open
└─ loop

```

```
| strip  
| print
```

12. Easy Edge Cases

Edge Case 1: Empty File

Suppose:

```
transactions.txt
```

contains nothing.

Then:

```
for line in file
```

has no lines to process.

Possible output:

```
No transaction output
```

For today's exercise, you do not necessarily need a special message unless you want to experiment.

Edge Case 2: File Not Found

Suppose your program expects:

```
transactions.txt
```

but the actual file is missing.

Python may produce:

```
FileNotFoundError
```

For now, simply understand why this happens.

Common causes include:

```
wrong filename  
wrong folder  
typing mistake  
file was never created
```

Later you will learn how to handle this with:

```
try  
except
```

13. Expected Input

Assume `transactions.txt` already exists.

Its content could be:

```
Deposit: 1000  
Withdraw: 300  
Payment: 200
```

Your function may receive the filename conceptually as:

```
transactions.txt
```

No keyboard input is required unless you choose to ask the user for the filename.

14. Expected Output

The program should print:

```
Deposit: 1000  
Withdraw: 300  
Payment: 200
```

It should **not** accidentally print:

```
Deposit: 1000  
  
Withdraw: 300  
  
Payment: 200
```

The goal is clean output with one transaction per line.

15. Hint Only

Start by thinking about this structure:

```
def read_transactions(filename):  
    with open(_____, "r") as file:  
        for _____ in file:  
            transaction = _____.strip()  
            print(_____)
```

Ask yourself four questions:

1. What filename should `open()` receive?
2. Which mode means read?
3. What variable represents each line?
4. What should `strip()` be applied to?

Today's mental model

```
transactions.txt
↓
open(..., "r")
↓
with
↓
for each line
↓
strip()
↓
print
```

Do not write the entire program immediately. First try completing the missing pieces in the hint yourself.